VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Faculty of Computer Science and Engineering

CC02 — Lab Report

Digital Signal Processing
Lab 1 – 2

Supervisors:
Students: Nguyen Dang Duy 2252116
Bui Thai An 2252001

Ho Chi Minh City, March 14, 2026

Contents

1 Lab 1 **2**
1.1 Overview . . . 2
1.2 Objectives . . . 2
1.3 Methodologies . . . 2
1.3.1 Hardware & Software . . . 2
1.4 Exercises . . . 2
1.4.1 Exercise 1.1 . . . 2
1.4.2 Exercise 1.2 . . . 4
1.5 Conclusion . . . 6

2 Lab 2 **7**
2.1 Overview . . . 7
2.2 Objectives . . . 7
2.3 Methodologies . . . 7
2.3.1 Hardware & Software . . . 7
2.4 Exercises . . . 7
2.4.1 Exercise 2.1 . . . 7
2.4.2 Exercise 2.2 . . . 7
2.4.3 Exercise 2.3 . . . 8
2.4.4 Exercise 2.4 . . . 9
2.4.5 Exercise 2.5 . . . 10
2.4.6 Exercise 2.6 . . . 11
2.4.7 Exercise 2.7 . . . 13
2.4.8 Exercise 2.8 . . . 15
2.5 Conclusion . . . 17

3 Reference **18**

1 Lab 1

1.1 Overview

This is the report of lab 1.

1.2 Objectives

- Install Scilab (https://www.scilab.org) if it was not installed in the computer.
- Read user manual: https://www.scilab.org/sites/default/files/Scilab_beginners.pdf
- Do additional exercises on SciLab
- Use SciLab to draw the signal after every step in the ADC procedure

1.3 Methodologies

1.3.1 Hardware & Software

- Hardware: Intel-based Laptop
- Software: Scilab (Windows OS)

1.4 Exercises

1.4.1 Exercise 1.1

Using the operators in vector and matrix to do the following tasks

- Create a vector in the form of $(x_1 + 1, x_2 + 1, x_3 + 1, x_4 + 1)$, where $x = [1, 2, 3, 4]$.
- Create a vector in the form of $(x_1y_1, x_2y_2, x_3y_3, x_4y_4)$, where $x = [1, 2, 3, 4]$ and $y = [5, 6, 7, 8]$.
- Create a vector in the form of $(\sin(x_1), \sin(x_2), \ldots, \sin(x_{10}))$, where x is a vector of 10 values linearly spaced in the interval $[0, \pi]$.

Answer

This is the source code of the exercise 1.1:

```
// Define x
x = [1:1:4];
vec_1 = x + 1
//Define y
y = [5:1:8];
vec_2 = x .* y
//Define z
z = linspace(0, %pi, 10);
vec_3 = sin(z)
```

The provided Scilab code performs the requested vector operations step-by-step:

- **Part 1:** `x = [1:1:4]` initializes the vector x with values from 1 to 4 using a step size of 1. The expression `vec_1 = x + 1` adds the scalar value 1 to each element of the vector x, outputting a new vector in the format $(x_1 + 1, x_2 + 1, x_3 + 1, x_4 + 1)$.
- **Part 2:** `y = [5:1:8]` initializes the vector y with values from 5 to 8. The expression `vec_2 = x .* y` utilizes the `.*` operator to perform element-wise multiplication between the corresponding elements of vectors x and y, producing $(x_1y_1, x_2y_2, x_3y_3, x_4y_4)$.
- **Part 3:** `z = linspace(0, %pi, 10)` generates a vector z containing 10 linearly spaced values starting at 0 and ending at π (using Scilab's built-in constant `%pi`). Finally, `vec_3 = sin(z)` applies the sine function element-wise to the vector z.

This is the result of the code:

```
--> // Define x

--> x = [1:1:4];

--> vec_1 = x + 1

 vec_1 = [1x4 double]

   2.   3.   4.   5.

--> //Define y

--> y = [5:1:8];

--> vec_2 = x .* y

 vec_2 = [1x4 double]

   5.   12.   21.   32.

--> //Define z

--> z = linspace(0, %pi, 10);

--> vec_3 = sin(z)

 vec_3 = [1x10 double]

         column 1 to 7

   0.   0.3420201   0.6427876   0.8660254   0.9848078   0.9848078   0.8660254

         column 8 to 10

   0.6427876   0.3420201   1.225D-16
```

Figure 1: *Exercise 1.1 result.*

1.4.2 Exercise 1.2

Consider the following analog signal $x_a(t)$:

- Use Scilab to draw $x_a(t)$ in 5 periods.
- Determine the discrete-time signal $x(n)$ of the signal $x_a(t)$ that is sampled with a sampling rate $F_s = 300$ samples/s.
- Determine the discrete-time signal $x(n)$ and determine the periodic property of $x(n)$. If $x(n)$ is a periodic signal, determine the frequency and period of $x(n)$. Then, use Scilab to draw $x(n)$ in 5 periods.
- Determine the quantized signal $x_q(n)$ if $\Delta = 0.1$ using the truncated method. Then, draw $x_q(n)$ in 5 periods.
- **Note:** All figures including $x_a(t)$, $x(n)$, and $x_q(n)$ must be displayed in a single window.

Answer

This is the source code of the Exercise 1.2:

```
// --- SETUP PARAMETERS ---
A = 3;
F0 = 50;
Fs = 300;
N = 6;                // Our calculated period
num_periods = 5;      // We need 5 periods for the assignment

// 1. DRAWING THE ANALOG SIGNAL (xa)
// We create 1000 points to make the line look  smooth  like an analog signal
t_analog = linspace(0, num_periods * (1/F0), 1000);
xa = A * sin(100 * %pi * t_analog);

// 2. DRAWING THE DISCRETE SIGNAL (x_n)
// Here, we only take points at integer steps (0, 1, 2...)
// 5 periods * 6 samples per period = 30 samples total
n = 0 : (num_periods * N);
x_n = A * sin(100 * %pi * n * (1/Fs));

// 3. DRAWING THE QUANTIZED SIGNAL (xq_n)
// 'fix' is the Scilab command for truncation (rounding toward zero)
xq_n = fix(x_n/delta)*delta;

// --- PLOTTING ---
clf(); // Clear any old drawings

// Subplot 1: Analog
subplot(3,1,1);
plot(t_analog, xa, 'r'); // 'r' makes the line red
xtitle( Analog Signal xa(t) ,  Time (s) ,  Amplitude );

// Subplot 2: Discrete
```

```
subplot(3,1,2);
plot2d3('gnn', n, x_n); // 'plot2d3' draws the vertical  stems
plot(n, x_n, 'ro');      // Adds red circles at the top of the stems
xtitle( Discrete Signal x(n) (Periodic N=6) ,  n (samples) ,  Amplitude );

// Subplot 3: Quantized
subplot(3,1,3);
plot2d3('gnn', n, xq_n);
plot(n, xq_n, 'bo');      // Adds blue circles
xtitle( Quantized Signal xq(n) (Truncated) ,  n (samples) ,  Amplitude );
```

Part 1, The period T of the signal is given by:

$$T = \frac{2\pi}{\omega} = \frac{2\pi}{100} = \frac{1}{50} \text{ (s)}$$

For 5 periods, the time range should be:

$$0 < t < 5T = \frac{5}{50} = 0.1 \text{ (s)}$$

Part 2, The discrete-time signal x(n) is obtained by sampling xa(t) at the interval $= nT_s$:

$$x(n) = x_a(nT_s)$$

Substituting $x_a(t) = 3sin(100\pi t)$:

$$x_a(t) = 3sin(100\pi(nT_s))$$

Since $T_s = \frac{1}{300}$:

$$x_a(t) = 3sin\left(100\pi \times \frac{n}{300}\right)$$

$$x_a(t) = 3sin\left(\frac{\pi n}{3}\right)$$

Part 3, From the sampling rate F_s and the period T of $x_a(t)$, we find that $N_0 = T \times F_s = 6$ samples. So in 5 periods, n ranges from 0 to $5N_0 = 30$ samples. Therefore, we can draw the plot for $x(n)$.

Part 4, In the truncated method, we discard excess digits. Hence, we define a truncate function $y = \text{truncate}(x)$ as $y = \text{sign}(x) \times \lfloor |x| \rfloor$ that does similar work where the absolute value of x is truncated and then multiplied by the sign of x. This is because $y = \lfloor x \rfloor$ alone produces an incorrect result when x is negative. Then, we can compute $x_q(n)$ by using the formula:

$$x_q(n) = \text{truncate}\left(\frac{n}{\Delta}\right) * \Delta$$

In this formula, Δn computes the number of quantization steps, then gets truncated to get an

integer value. We then multiply it back with Δ. This is the plot for $x_q(n)$ in 5 periods.

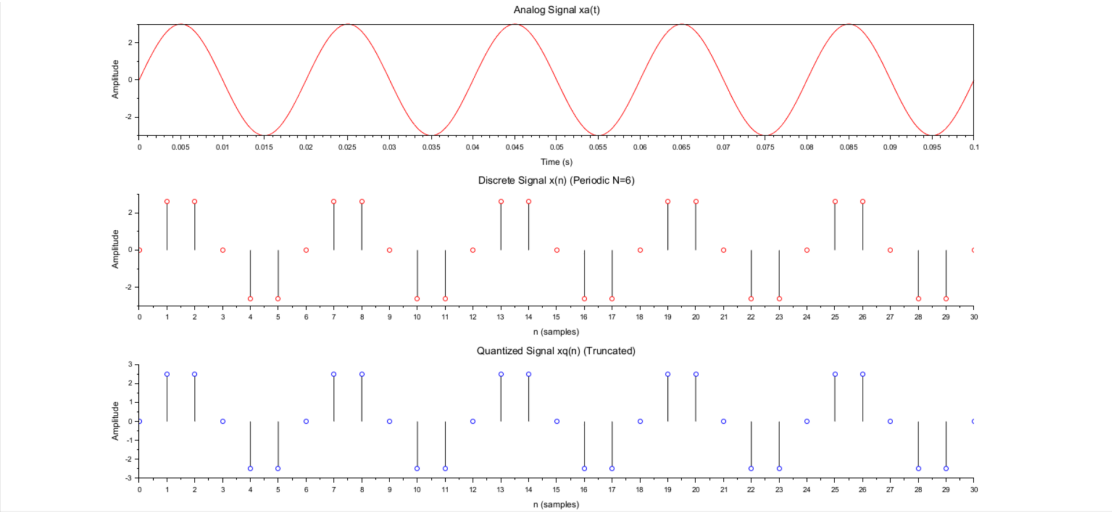

Figure 2: *Analog Signal, Discrete Signal and Quantitized Signal for Exercise 1.2*

1.5 Conclusion

- Learn basic how to use Scilab

- Use SciLab to draw the signal after every step in the ADC procedure

2 Lab 2

2.1 Overview

This is the report of lab 2.

2.2 Objectives

- Review lecture notes of discrete-time signal and system
- Do additional exercises on SciLab

2.3 Methodologies

2.3.1 Hardware & Software

- Hardware: Intel-based Laptop
- Software: Scilab (Windows OS)

2.4 Exercises

2.4.1 Exercise 2.1

Let investigate the following functions on Scilab and briefly report their functionalities and how to use it.

Functions	**Description (will be filled by students)**
plot2d3(...)	Plots a 2D discrete-time signal with a predefined style.
min(...)	Returns the minimum value in a vector or matrix
max(...)	Returns the maximum value in a vector or matrix
subplot(...)	Divides a figure window into multiple subplots for displaying multiple plots.
title(...)	Adds a title to the current plot
xlabel(...)	Adds a labels the x-axis of the current plot.
ylabel(...)	Adds a labels the y-axis of the current plot.
bool2s(...)	Converts a boolean (logical) vector into a numerical signal (where true becomes 1, false becomes 0).
deff(...)	Defines a function inline, which is useful for quick function definitions

2.4.2 Exercise 2.2

Try the following scripts on Scilab and report what your understanding after observing the output.

```
scilab:> n = -5:5;
```

```
scilab:> msignal = bool2s (n >= 0);
scilab:> plot2d3(n, msignal)
```

Answer

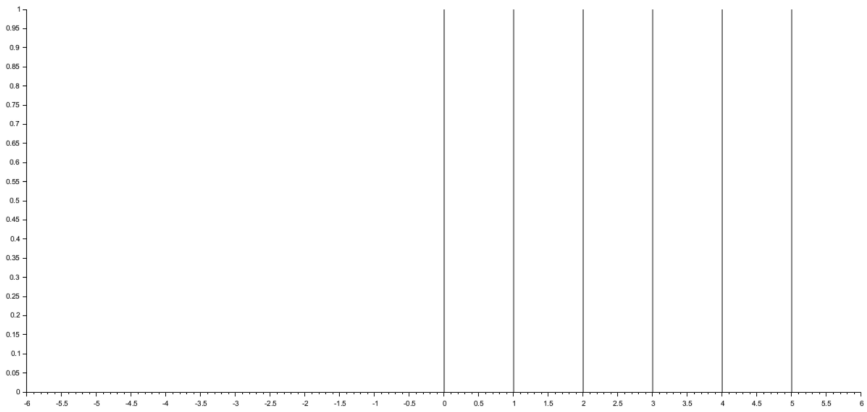

Figure 3: *Exercise 2.2 output*

The condition `n >= 0` produces a boolean vector and then the function bool2s converts this into numerical signal:

- For `n < 0`, `msignal` is 0, so there are no visible lines.
- For `n >= 0`, `msignal` is 1, so there are vertical lines at n = 0, 1, 2, 3, 4, 5,...

The output is a discrete-time unit step signal.

2.4.3 Exercise 2.3

Try the following scripts on Scilab and report what your understanding after observing the output.

```
n = -5:5;
msignal = bool2s (n == 0);
plot2d3(n, msignal)
```

Answer

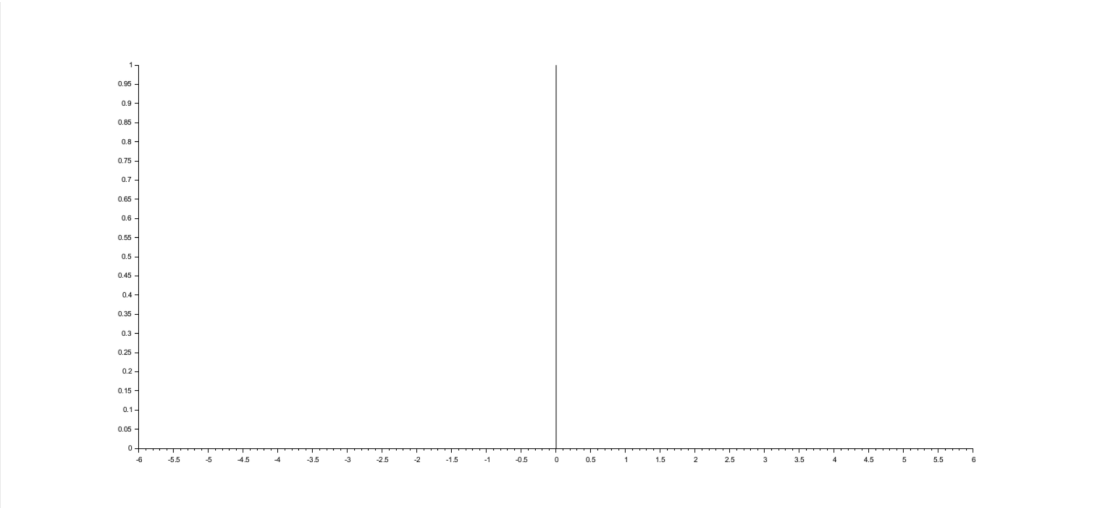

Figure 4: *Exercise 2.3 Output*

Since `msignal` is 1 at n = 0, so there is only 1 line in the plot

2.4.4 Exercise 2.4

Use Scilab to draw the unit ramp signal $u_r(n)$ for $n = -5 : 5$

Answer

This is the source code for Exercise 2.4:

```
n = -5 : 5;
msignal = n.*bool2s (n >= 0);
plot2d3(n, msignal)
```

The unit ramp signal $u_r(n)$ is defined as:

$$u_r(n) = \begin{cases} n, & n \geq 0 \\ 0, & n \leq 0 \end{cases}$$

The signal increases linearly for $n \geq 0$ and is zero when $n < 0$

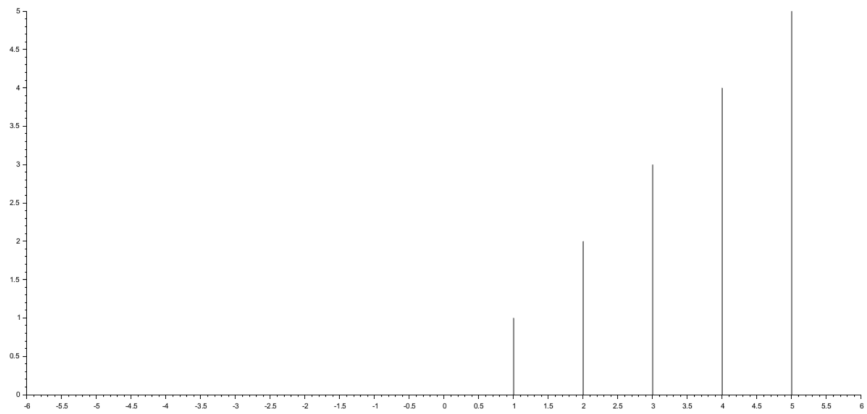

Figure 5: *Exercise 2.4 Output.*

2.4.5 Exercise 2.5

Given a discrete-time signal $x(n) = \{1, \underset{\uparrow}{3}, -2\}$. Use Scilab to draw the signal $x(n)$, the odd signal component $x_0(n)$, and the even signal component xe(n). Each signal will be drawed by a single plot but they are displayed in a single window. Please use `title()`, `xlabel()` and `ylabel()` to represent the name of each plot.

Answer

This is the source code for Exercise 2.5:

```
// 1. Define the signal and time index
n = -1:1;
x = [1, 3, -2];

// 2. Calculate the flipped signal x(-n)
// we use -n as the index to ensure the flip is relative to n=0
x_flipped = [x(3), x(2), x(1)];

// 3. Calculate Even and Odd components
xe = (x + x_flipped) / 2;
xo = (x - x_flipped) / 2;

// 4. Plotting
clf();

// Original Signal
subplot(3,1,1);
plot2d3('gnn', n, x);
plot(n, x, 'ro');
title( Original Signal x(n) = {1, 3, -2} );
xlabel( n );
ylabel( Amplitude );
```

```
gca().children.children.thickness=3;
xgrid();

// Even Component
subplot(3,1,2);
plot2d3('gnn', n, xe);
plot(n, xe, 'bo');
title( Even Component xe(n) );
xlabel( n );
ylabel( Amplitude );
gca().children.children.thickness=3;
xgrid();

// Odd Component
subplot(3,1,3);
plot2d3('gnn', n, xo);
plot(n, xo, 'go');
title( Odd Component xo(n) );
xlabel( n );
ylabel( Amplitude );
gca().children.children.thickness=3;
xgrid();
```

A signal $x(n)$ can be decomposed into even and odd components as:

$$x_e(n) = \frac{x(n) + x(-n)}{2}$$

$$x_o(n) = \frac{x(n) - x(-n)}{2}$$

Since $x(n)$ is only defined at three points, we assume symmetry around $n = 0$ and extend the signal symmetrically:

$$x(-n) = \{1, 3, -2\}$$

Then, using the formulas above:

$$x_e(n) = \frac{\{1, 3, -2\} + \{-2, 3, 1\}}{2} = \frac{\{-1, 6, -1\}}{2} = \{-0.5, 3, -0.5\}$$

$$x_o(n) = \frac{\{1, 3, -2\} - \{-2, 3, 1\}}{2} = \frac{\{3, 0, -3\}}{2} = \{1.5, 0, -1.5\}$$

2.4.6 Exercise 2.6

Given two discrete-time signals $x_1(n) = \{\underset{\uparrow}{0}, 1, 3, -2\}$ and $x_2(n) = \{0, \underset{\uparrow}{1}, 2, 3\}$

Determine $y(n) = x_1(n) + x_2(n)$. Then use Scilab to draw $x_1(n)$, $x_2(n)$ and $y(n)$. Each signal will be drawed by a single plot but they are displayed in a single window. Please use `title()`, `xlabel()` and `ylabel()` to represent the name of each plot.

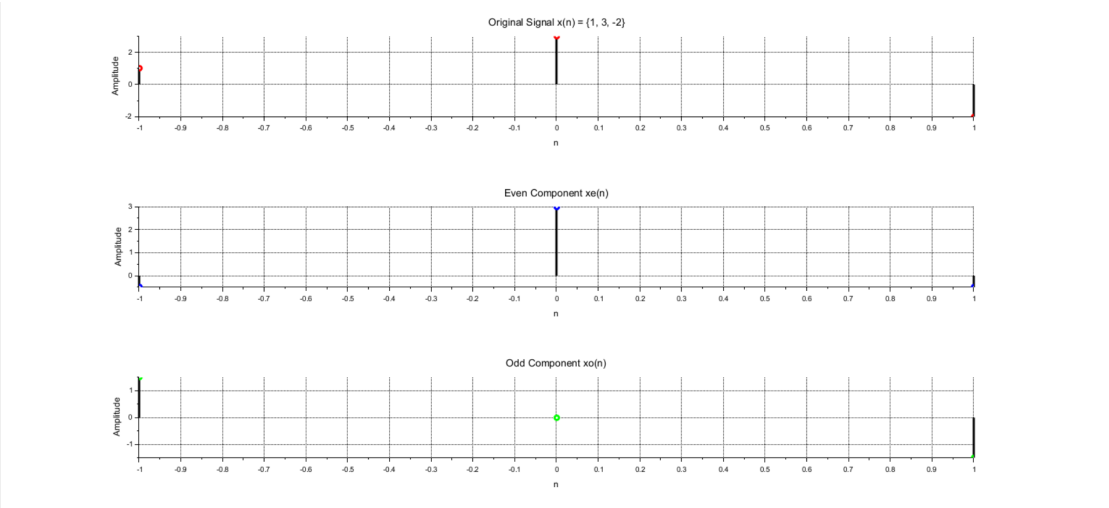

Figure 6: *Output for Exercise 2.5*

Answer

This is the code for exercise 2.6:

```
clc();
// 1. Define the common time index
n = -1 : 3;

// 2. Define signals (padded with zeros to match the length of n)
// n: -1  0  1  2   3
x1 = [ 0, 0, 1, 3, -2 ];
x2 = [ 0, 1, 2, 3,  0 ];

// 3. Perform addition
y = x1 + x2;

// --- Plotting ---
clf();

// Plot x1(n)
subplot(3,1,1);
plot2d3('gnn', n, x1);
plot(n, x1, 'ro');
title( Signal x1(n) = {0, 1, 3, -2} );
xlabel( n );
ylabel( Amplitude );

// Plot x2(n)
subplot(3,1,2);
plot2d3('gnn', n, x2);
plot(n, x2, 'bo');
title( Signal x2(n) = {0, 1, 2, 3} );
xlabel( n );
```

```
ylabel( Amplitude );

// Plot y(n)
subplot(3,1,3);
plot2d3('gnn', n, y);
plot(n, y, 'go');
title( Result y(n) = x1(n) + x2(n) );
xlabel( n );
ylabel( Amplitude );

// Add grids for better visibility
for i=1:3; subplot(3,1,i); xgrid(); end
```

We can explicitly write the values of $x_1(n)$ and $x_2(n)$ and add them one by one to determine $y(n)$:

n	**-1**	**0**	**1**	**2**	**3**
$x_1(n)$	0	0	1	3	-2
$x_2(n)$	0	1	2	3	0
$y(n)$	0	1	3	6	-2

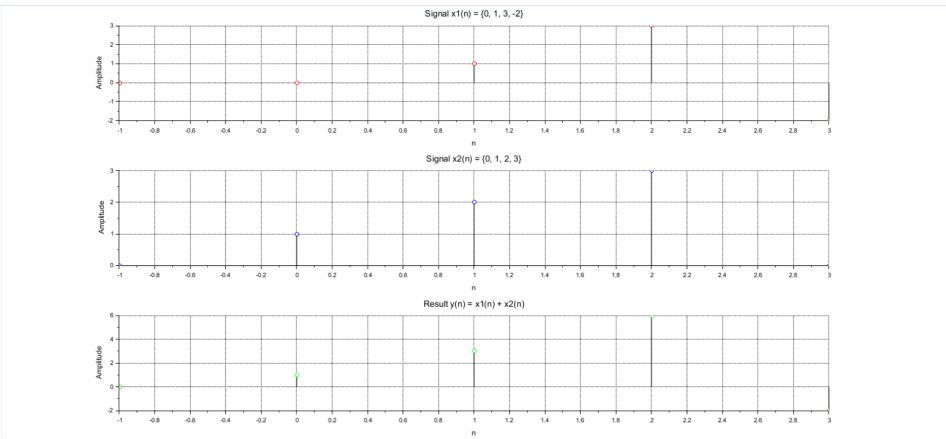

Figure 7: *Output for Exercise 2.6*

2.4.7 Exercise 2.7

Given two discrete-time signals $x_1(n) = \{\underset{\uparrow}{0}, 1, 3, -2\}$ and $x_2(n) = \{0, \underset{\uparrow}{1}, 2, 3\}$

Determine $y(n) = x_1(n).x_2(n)$. Then use Scilab to draw $x_1(n)$, $x_2(n)$ and $y(n)$. Each signal will be drawed by a single plot but they are displayed in a single window. Please use `title()`, `xlabel()` and `ylabel()` to represent the name of each plot.

Answer

This is the source code of Exercise 2.7:

```
clc();
```

```
// 1. Define the common time index
n = -1 : 3;

// 2. Define signals (padded with zeros to match the length of n)
// n: -1  0  1  2   3
x1 = [ 0, 0, 1, 3, -2 ];
x2 = [ 0, 1, 2, 3,  0 ];

// 3. Perform addition
y = x1 .* x2;

// --- Plotting ---
clf();

// Plot x1(n)
subplot(3,1,1);
plot2d3('gnn', n, x1);
plot(n, x1, 'ro');
title( Signal x1(n) = {0, 1, 3, -2} );
xlabel( n );
ylabel( Amplitude );

// Plot x2(n)
subplot(3,1,2);
plot2d3('gnn', n, x2);
plot(n, x2, 'bo');
title( Signal x2(n) = {0, 1, 2, 3} );
xlabel( n );
ylabel( Amplitude );

// Plot y(n)
subplot(3,1,3);
plot2d3('gnn', n, y);
plot(n, y, 'go');
title( Result y(n) = x1(n) * x2(n) );
xlabel( n );
ylabel( Amplitude );

// Add grids for better visibility
for i=1:3; subplot(3,1,i); xgrid(); end
```

We can explicitly write the values of $x_1(n)$ and $x_2(n)$ and add them one by one to determine $y(n)$:

n	**-1**	**0**	**1**	**2**	**3**
$x_1(n)$	0	0	1	3	-2
$x_2(n)$	0	1	2	3	0
$y(n)$	0	0	2	9	0

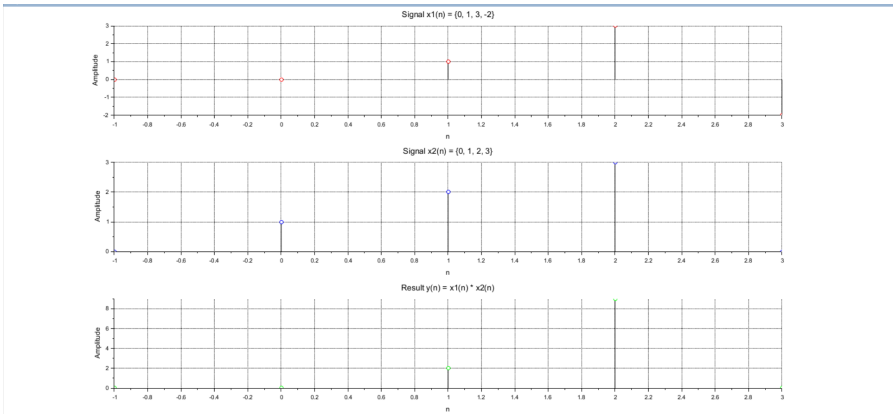

Figure 8: *Output for exercise 2.7*

2.4.8 Exercise 2.8

Given a discrete-time signal $x(n) = \{1, -2, \underset{\uparrow}{3}, 6\}$ Determine the following signal and then use Scilab to draw the original signal $x(n)$ and the manipulated signal $yi(n)$. Each pair of plots will be display in a single window. Please use `title()`, `xlabel()` and `ylabel()` to represent the name of each plot.

- $y_1(n) = x(-n)$
- $y_2(n) = x(n+3)$
- $y_3(n) = 2x(-n-2)$

Answer

This is the source code for exercise 2.8:

```
clc();
// --- Define Original Signal ---
n_orig = -2:1;
x = [1, -2, 3, 6];

// --- Case a: y1(n) = x(-n) ---
n1 = -1:2;
y1 = [x(4), x(3), x(2), x(1)]; // Flipped order

// --- Case b: y2(n) = x(n + 3) ---
n2 = n_orig - 3; // Shift left means subtracting from index
y2 = x;

// --- Case c: y3(n) = 2x(-n - 2) ---
// We flip the original n and then shift it
n3 = 1:4;
y3 = [x(4), x(3), x(2), x(1)]; // Amplitude scaling
y3 = 2.*y3

// --- Plotting ---
clf();
```

```

// Row 1: Case a
subplot(3,2,1);
plot2d3('gnn', n_orig, x);
plot(n_orig, x, 'ro');
title( Original x(n) );
xlabel( n );
ylabel( Amp ); xgrid();

subplot(3,2,2);
plot2d3('gnn', n1, y1);
plot(n1, y1, 'bo');
title( y1(n) = x(-n) );
xlabel( n );
ylabel( Amp ); xgrid();

// Row 2: Case b
subplot(3,2,3);
plot2d3('gnn', n_orig, x);
plot(n_orig, x, 'ro');
title( Original x(n) );
xlabel( n ); ylabel( Amp );
xgrid();

subplot(3,2,4);
plot2d3('gnn', n2, y2);
plot(n2, y2, 'bo');
title( y2(n) = x(n+3) );
xlabel( n );
ylabel( Amp );
xgrid();

// Row 3: Case c
subplot(3,2,5);
plot2d3('gnn', n_orig, x);
plot(n_orig, x, 'ro');
title( Original x(n) );
xlabel( n );
ylabel( Amp );
xgrid();

subplot(3,2,6);
plot2d3('gnn', n3, y3);
plot(n3, y3, 'bo');
title( y3(n) = 2x(-n-2) );
xlabel( n );
ylabel( Amp );
xgrid();
```

This is the output for exercise 2.8:

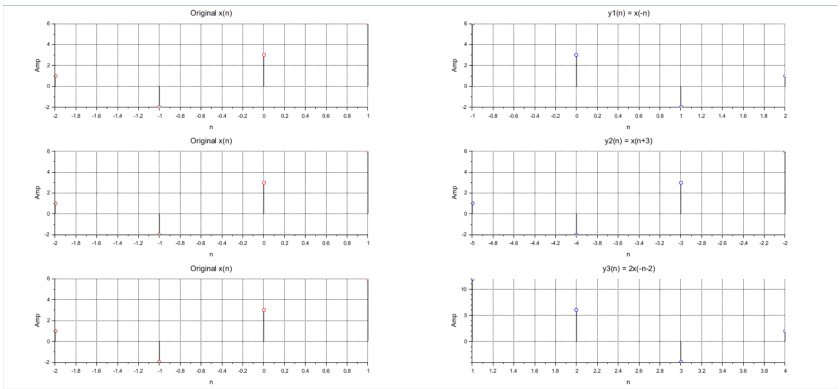

Figure 9: *Enter Caption*

2.5 Conclusion

- Learn basic functions on Scilab and their functionalities

3 Reference

The full source code is available in the GitHub repository: https://github.com/CoderMongmer/DSP-LAB-12